EXIT TICKET / REFLECTION

# Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

**In C#, == is used to compare two values for equality.**

**How confident are you? Circle one: Confident / Getting there / Need help**

Explain your choice, identifying one concept you can now use independently.

**GREEN PEN — Correct and explain**

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.





DO NOW — RECALL

# Sequence & branching in C#

Name: _____ Class: _____ Date: _____

**1. Why would Unicode be chosen instead of standard ASCII for a multilingual application?**

**2. Convert 01011000 0011 to denary using an 8-bit two’s-complement mantissa and a 4-bit two’s-complement exponent.**

**3. Convert $6D_{16}$ to eight-bit binary.**

**4. Convert 11110110 from 8-bit two’s-complement binary to denary. Show signed place values.**

DO NOW — CHECK / OBJECTIVES

# Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

**1. Model answer**

Unicode supports a much larger repertoire of characters from many writing systems. Standard ASCII has 128 codes and is centred on a limited character set. Unicode provides code points for many languages and symbols.

**2. Model answer**

5.5 Mantissa 01011000 = 0.6875 ; exponent 0011 = 3 ; $0.6875 \times 2^3 = 5.5$ .

**3. Model answer**

$01101101_2$ 6 → 0110 and D → 1101 .

**4. Model answer**

−10 −128 + 64 + 32 + 16 + 4 + 2 = −10 .

**By the end, you can…**

• identify sequence and branching as programming constructs;

• write input, output and simple casts using the C# syntax in your .NET Fiddle guide;

• write and test if / else if / else decisions using suitable conditions.

**Vocabulary — know these terms**

Recall the term — Sequence

Recall the term — Branching / selection

Recall the term — Boolean expression

Recall the term — Condition

Recall the term — Relational operators

Recall the term — Logical operators

Recall the term — Parsing

Recall the term — Boundary test data





YOU DO — INDEPENDENT PRACTICE

# Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

**Question 3**

Delivery charge
Ask for an order total and whether the customer is a member.
Output Free delivery only when the total is at least 30 AND the member answer is "yes" . Otherwise output Delivery charge applies .

**Question 4**

Simple login
Ask for a username and a PIN.
Output Access granted only when the username is "admin" and the PIN is 2468 .
Otherwise output Access denied .

YOU DO — INDEPENDENT PRACTICE

# Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

### Question 1

Positive, negative or zero
Ask the user for an integer. Output exactly one of:
Positive · Negative · Zero

### Question 2

Ticket category
Ask for an age and output:
Child for 0–11 · Teen for 12–17 · Adult for 18+





NEW LEARNING

# Read, annotate and explain

### Use the guide syntax

```
Console.Write("Prompt: ");
string text = Console.ReadLine();

int number =
int.Parse(Console.ReadLine());

if (condition)
{
// instructions
}
else if (condition)
{
// instructions
}
else
{
// instructions
}
```

OCR constructs: sequence · branching/selection · iteration. A past OCR question expects the construct name and a working example.

### Sequence = instructions in order

Sequence is the normal top-to-bottom execution of statements. Each statement happens after the previous one unless another construct changes the flow.

### Input arrives as text

Console.ReadLine() returns a string. If the program needs a whole number, parse the input into an int .

### Branching chooses a route

An if condition must evaluate to true or false . Only the matching branch is executed.

### Conditions and order matter

Use == to compare equality, not = . Use && , || and ! for AND, OR and NOT.

With else if , conditions are tested from top to bottom. Put the most demanding threshold first.

### Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

# Follow the worked method

### Build a result checker

### Use the standard .NET Fiddle structure

```
using System;

public class Program
{
    public static void Main()
    {
        // code goes here
    }
}
```

Main program instructions belong inside Main() .

### Read the score as an integer

```
Console.Write("Enter score: ");
int score =
    int.Parse(Console.ReadLine());
```

Console.ReadLine() gives us text; int.Parse(...) converts it to a whole number.

### Test the highest threshold first

```
if (score >= 70)
{
    Console.WriteLine("Distinction");
}
else if (score >= 50)
{
    Console.WriteLine("Pass");
}
else
{
    Console.WriteLine("Not yet");
}
```

A score of 85 satisfies both >= 70 and >= 50 , so the higher threshold must be tested first.

### Use values that expose mistakes

```
85 → Distinction
70 → Distinction
50 → Pass
49 → Not yet
```

Repeat the method with different inputs in your notebook. Check every intermediate value.





WE DO — GUIDED PRACTICE

# Try → Discuss → Improve

### 1. Even or odd

Ask for a whole number. Output Even when the remainder after division by 2 is zero; otherwise output Odd .

```
Console.Write("Number: ");
int number = int.Parse(Console.ReadLine());

// ?
```

Method / working:

### 2. Three temperature bands

Output Hot for 25+, Mild for 15–24, otherwise Cold .

Method / working:

### 3. Age range using AND

Output Teen ticket only when the age is from 12 to 17 inclusive.

Method / working:

### 4. Name it and show it

A past OCR programming-techniques question expects the three constructs to be named and supported with a working example . Today: give a C# example of sequence and a C# example of branching . Iteration comes next lesson.

Method / working: